Problem Set 6

Due March 6, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. **Point Cloud Alignment:** Consider two point clouds $(r_1, \ldots, r_N)$ and $(\hat{r}_1, \ldots, \hat{r}_N)$, where $r_i, \hat{r}_i \in \mathbb{R}^3$ for all i. We would like to translate and rotate the second point cloud so that it best matches with the first. Assume both clouds have the same number of points and we know which point in the first cloud corresponds to which point in the second (these assumptions greatly simplify the problem and are the focus of current research). We would like to find a rotation R and translation τ that minimizes the following loss

$$J(R, \tau) = \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2$$

subject to

$$RR^T = I$$

This is a non-convex optimization problem due to the orthogonality constraint $RR^T = I$. However, we can still solve this problem optimally. Let's seek to solve this optimization problem through two ways.

   (a) Solve for the optimal $\tau*$ as a function of $R, \hat{r}_i, r_i$. This should just be the solution to an ordinary least-squares problem. In terms of properties of the two point clouds, what does τ physically represent?

   (b) Solve for the rotation R^* by treating it as a linear optimization problem. In the process, also show that the rotation matrix you solved is globally optimal (i.e. there doesn't exist another rotation matrix that can have a lower loss).

   Hint: The Singular Value Decomposition $A = U\Lambda V^T$, where U, V are orthogonal and Λ is diagonal, may come in handy. All matrices have an SVD decomposition.

   (c) Show that your answer in part (b) is equivalent to taking the SVD of a simpler 3×3 matrix $H = \sum_j^N (\hat{r} - \hat{r}_c)(r - r_c)^T$.

   Remark: Part (c) is important because you will need a fast implementation of getting the rotation for Problem 3.

   (d) Align two point clouds, one the ground-truth `ground-truth.ply` and the other, a rotated, translated point cloud with additive noise `transformed-associated.ply`. What's the loss $J(R_t^*, \tau_t^*)$?

   (e) What's the SE(3) error between your optimized pose and the ground-truth pose (located in `gt-pose.csv`)? The CSV file stores the pose as a quaternion (x, y, z, w) and the translation (px, py, pz) in that order. Report both the rotational error (in degrees), and the translational error.

   (f) Is it possible that the program is optimally solved, but the R matrix of the solution does not correspond to a physical rotation? In which case, what does R physically do to the point cloud? (Aside: even when R is not a rotation matrix, there are methods to still extract a valid rotation).

2. **Bundle Adjustment SLAM in 2D:** In this problem you will use a standard open source SLAM package, GTSAM, to perform a bundle adjustment SLAM. (Recall, bundle adjustment SLAM solves for both landmark locations and robot poses at the same time. Alternatively, Pose Graph Optimization (PGO) SLAM pre-processes raw sensor measurements in a "front end", and optimizes only robot poses in a "back end.") We will use the Python interface to the GTSAM library, which is implemented in C++. We use a sparse 2D problem setup to ensure quick optimization in Google Colab. If you choose to consider dense or 3D SLAM for your final project, you will likely not want to use Google Colab, and you would likely use a PGO style SLAM.

(a) Draw a simple factor graph by hand or with a computer program, such as PowerPoint, that meets the following criteria.

- Three pose states x_0, x_1, and x_2.
- Two landmarks, ℓ_0 and ℓ_1, marked by a 2D position.
- Four range and bearing measurements as follows:
  - $y^{rb}_{0,0}$: between pose x_0 and landmark ℓ_0
  - $y^{rb}_{0,1}$: between pose x_0 and landmark ℓ_1
  - $y^{rb}_{1,1}$: between pose x_1 and landmark ℓ_1
  - $y^{rb}_{2,1}$: between pose x_2 and landmark ℓ_1
- Odometry measurements $y^{o}_{i,i+1}$ between successive poses.
- Loop closure constraint as odometry-like measurement $y^{lc}_{0,2}$ between pose states x_0 and x_2.

We now consider a more complex scenario. We will use the robot trajectory and landmarks in Figure 1. You will see three data files on Canvas. In the file `robot_history_5.csv`, we provide the ground truth history of the robot pose and measurements for a maximum sensor range of 5 meters. We will add noise and reconstruct the robot pose and landmark locations. The following steps will guide you in constructing the factor graph in GTSAM.

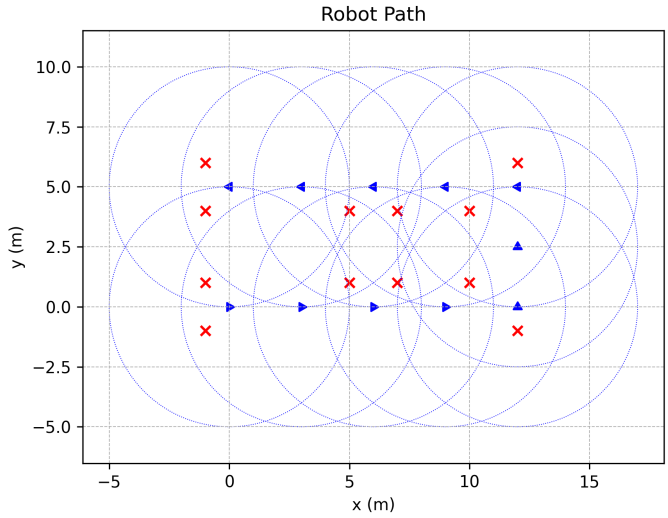


Figure 1: Robot Ground Truth Path. The landmarks are indicated as a red '×', the robot is a blue triangle oriented in the direction of travel, and the dotted blue circles indicate the maximum sensor range.

Please refer to the provided Google Colab at the link below (using your Stanford login), though you do not need to use Google Colab if you prefer to install GTSAM locally.

`https://colab.research.google.com/drive/1tJqy1jNUeGC2V235yTdWqyOElQdIclMp?usp=sharing`

(b) Verify that you can pip install GTSAM. On Google Colab, you can run `!pip install gtsam` at the top. Please read through the provided code to load and process the files. Instantiate the factor graph with:

```
graph = gtsam.NonlinearFactorGraph()
```

This is an empty factor graph. To *add* a factor to the factor graph, we will call `graph.add(...)`. To simplify the notation, GTSAM offers shorthand variables `X` and `L` for state and landmark, respectively, to organize the SLAM factors. In the provided Google Colab, we have initialized lists for `X` and `L`. The first factor we want to add is the prior factor to initialize the map at the origin. Fill in the required code based on the factor description below.

`gtsam.PriorFactorPose2(X_start, pose, noise)`

For `X_start`, use the zero-th element of the `X` list. For `pose`, use the `gtsam.Pose2(x, y, theta)` function specifying the origin and zero angle (i.e., aligned with the x-axis).

(c) Iterate through the provided data (`Pandas` is recommended). Use the factor type below to add the relative motion factors (i.e., from the odometry or relative state between successive states).

`gtsam.BetweenFactorPose2(X_prev, X_curr, rel_pose, motion_noise)`

Use the factor type below to add the bearing and range factors (i.e., from the sensor model) per each landmark in view at the given time step.

`gtsam.BearingRangeFactor2D(X_curr, L_curr, bearing, range, noise`

For the bearing measurement, use `gtsam.Rot2.fromDegrees(bearing_angle)`. Lastly, we will use the `BetweenFactorPose2` method to add a loop closure constraint between the second and eighth elements (zero-indexed) of the `X` list with position offset of 0 horizontally, 5 vertically, and 180 degrees in rotation. How many factors are in your factor graph (print `graph` to verify)?

(d) We will initialize the values from the motion alone. Instantiate the initial estimate with

`initial_estimate = gtsam.Values()`

You can input an initial estimate with `initial_estimate.insert(X_or_L_var, pose_or_point)`. Be sure to use `gtsam.Pose2(x, y, theta)` for the pose and `gtsam.Point2(x, y)` for the landmark. Plot the initial estimates for `X` and `L` using the functions below:

`gtsam_plot.plot_pose2(...)` and `gtsam_plot.plot_point2(...)`.

Include the plot in your write-up.

(e) Run the Levenberg Marquardt in GTSAM as listed in the Google Colab. GTSAM will provide estimates of `X` and `L` through the optimizer and will provide the marginal Gaussian distributions in `gtsam.Marginals`. Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma). Include the plot in your write-up.

(f) What do you notice about the orientation of the solution and the shape and orientation of the confidence ellipses? Why does this occur?

(g) Switch out `robot_history_5.csv` for `robot_history_3.csv` (maximum sensor range of 3 meters) and `robot_history_10.csv` (maximum sensor range of 10 meters). Using the same initialization for `X` and `L`, reconstruct the factor graphs. What are the factor graph sizes in each case?

Hint: You should only have to change the file input. All the other functions you wrote should still be applicable!

(h) Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma) in each case. What do you notice about the final results at different sensor ranges? Explain your result with the connectivity of the resulting factor graphs.

1. **Point Cloud Alignment:** Consider two point clouds $(r_1, \ldots, r_N)$ and $(\hat{r}_1, \ldots, \hat{r}_N)$, where $r_i, \hat{r}_i \in \mathbb{R}^3$ for all i. We would like to translate and rotate the second point cloud so that it best matches with the first. Assume both clouds have the same number of points and we know which point in the first cloud corresponds to which point in the second (these assumptions greatly simplify the problem and are the focus of current research). We would like to find a rotation R and translation τ that minimizes the following loss

$$J(R, \tau) = \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2$$

subject to

$$RR^T = I$$

This is a non-convex optimization problem due to the orthogonality constraint $RR^T = I$. However, we can still solve this problem optimally. Let's seek to solve this optimization problem through two ways.

(a) Solve for the optimal $\tau*$ as a function of $R, \hat{r}_i, r_i$. This should just be the solution to an ordinary least-squares problem. In terms of properties of the two point clouds, what does τ physically represent?

(b) Solve for the rotation R^* by treating it as a linear optimization problem. In the process, also show that the rotation matrix you solved is globally optimal (i.e. there doesn't exist another rotation matrix that can have a lower loss).
Hint: The Singular Value Decomposition $A = U\Lambda V^T$, where U, V are orthogonal and Λ is diagonal, may come in handy. All matrices have an SVD decomposition.

(c) Show that your answer in part (b) is equivalent to taking the SVD of a simpler 3×3 matrix $H = \sum_j^N (\hat{r} - \hat{r}_c)(r - r_c)^T$.
Remark: Part (c) is important because you will need a fast implementation of getting the rotation for Problem 3.

(d) Align two point clouds, one the ground-truth `ground-truth.ply` and the other, a rotated, translated point cloud with additive noise `transformed-associated.ply`. What's the loss $J(R_t^*, \tau_t^*)$?

(e) What's the SE(3) error between your optimized pose and the ground-truth pose (located in `gt-pose.csv`)? The CSV file stores the pose as a quaternion (x, y, z, w) and the translation (px, py, pz) in that order. Report both the rotational error (in degrees), and the translational error.

(f) Is it possible that the program is optimally solved, but the R matrix of the solution does not correspond to a physical rotation? In which case, what does R physically do to the point cloud? (Aside: even when R is not a rotation matrix, there are methods to still extract a valid rotation).

a) $J(R,\tau) = \frac{1}{N}\sum_{i=1}^{N} \left\| \left(R\hat{r}_i + \tau - r_i\right) \right\|^2$

$e_i = R\hat{r}_i + \tau - r_i$

$J(R,\tau) = \frac{1}{N}\sum_{i=1}^{N} \|e\|^2 = \frac{1}{N}\sum_{i=1}^{N} e_i^T e_i$

$\frac{\partial J}{\partial \tau} = \frac{1}{N}\sum_{i=1}^{N} 2e_i \frac{\partial e}{\partial \tau} = \frac{1}{N}\sum_{i=1}^{N} 2e_i = 0$

$\sum_{i=1}^{N} e_i = \sum_{i=1}^{N} R\hat{r}_i + \tau - r_i = 0$

$\sum_{i=1}^{N} R\hat{r}_i + \sum_{i=1}^{N} \tau^* + \sum_{i=1}^{N} r_i = 0 \quad = R\sum_{i=1}^{N} \hat{r}_i + N\tau^* - \sum_{i=1}^{N} r_i = 0$

$N\tau^* = \sum_{i=1}^{N} r_i - R\sum_{i=1}^{N} \hat{r}_i$

$\boxed{\tau^* = r_c - R\hat{r}_c}$

τ aligns the 2 point clouds after rotation and τ^* aligns them with the smallest loss

c) $J(R,\tau^*) = \frac{1}{N}\sum_{i=1}^{N} \left\| \left(R\hat{r}_i + \tau^* - r_i\right) \right\|^2 = \frac{1}{N}\sum_{i=1}^{N} \|R\bar{r}_i - \bar{r}_i\|$

$= \frac{1}{N}\sum_{i=1}^{N} \left[\left(R\bar{r}_i\right)^T \left(R\bar{r}_i\right) + \bar{r}_i^T\bar{r}_i - 2\left(R\hat{r}_i\right)^T \bar{r}_i \right]$

$\hat{r}_i^T R^T R \hat{r}_i = \hat{r}_i^T \hat{r}_i$ (I)

$\bar{r}_i^T R^T \bar{r}_i$

$J(R,\tau^*) = \frac{1}{N}\sum_{i=1}^{N} \left[\hat{r}_i^T\hat{r}_i + \bar{r}_i^T\bar{r}_i - 2\hat{r}_i^T R^T \bar{r}_i \right]$

minimizing J same as maximizing this term (because of negative sign)

Fixing $U\Lambda V^T$ → $a^T b = tr(ab^T)$ → $\sum \bar{r}_i^T R^T \bar{r}_i = tr\left(R\Sigma \hat{r}_i \bar{r}_i^T\right)$

H matrix where $H = U\Lambda U^T$

$tr(RH) = tr\left(R U\Lambda V^T\right) = tr\left(V^T R U \Lambda\right)$

all orthogonal; Q

for orthogonal matrices, $\|q_{ii}\| \leq 1$, so max when $q_{ii} = 1$ ∴ $Q = I$

∴ $Q = V^T R U$ $\quad R = V U^T$

$\boxed{R^* = VU^T}$

globally optimal because we set all $q_{ii} = 1$ and no matrix can produce a larger trace

c) $H = \sum_{j=1}^{N} (\hat{v}_j \cdot \hat{v}_c)(r - r_c)^T = \sum_{j=1}^{N} \bar{v}_j r_j^T$ from before

SVD $= U \Lambda V^T$ like before

use H to get U and V^T, then get $R^* = VU^T$

d)

```
Optimal translation : [0.51769478 0.57728611 0.50401143]
Optimal rotation:
[[-0.30702737 -0.9514257   0.02287657]
 [-0.89566072  0.28073815 -0.34493183]
 [ 0.32175468 -0.12639316 -0.93834892]]
Loss J(R, tau): 0.007494166321482214
```

e)

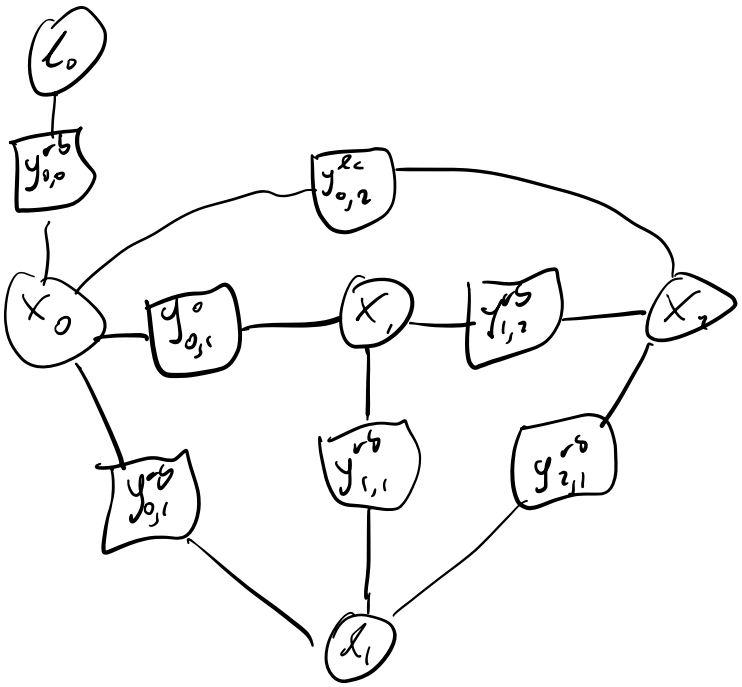
```
Rotation error: 6.168882851498211e-15º
Translation error: 1.6011864169946884e-15m
```

f) Since $R^T R = I$ we know $\det(R) = \pm 1$ but $\det(R) = -1$ flips the image, which is not physically possible. This could be handled by flipping the sign of one vector

2. **Bundle Adjustment SLAM in 2D:** In this problem you will use a standard open source SLAM package, GTSAM, to perform a bundle adjustment SLAM. (Recall, bundle adjustment SLAM solves for both landmark locations and robot poses at the same time. Alternatively, Pose Graph Optimization (PGO) SLAM pre-processes raw sensor measurements in a "front end", and optimizes only robot poses in a "back end.") We will use the Python interface to the GTSAM library, which is implemented in C++. We use a sparse 2D problem setup to ensure quick optimization in Google Colab. If you choose to consider dense or 3D SLAM for your final project, you will likely not want to use Google Colab, and you would likely use a PGO style SLAM.

(a) Draw a simple factor graph by hand or with a computer program, such as PowerPoint, that meets the following criteria.

- Three pose states x_0, x_1, and x_2.
- Two landmarks, ℓ_0 and ℓ_1, marked by a 2D position.
- Four range and bearing measurements as follows:
  - $y^{rb}_{0,0}$: between pose x_0 and landmark ℓ_0
  - $y^{rb}_{0,1}$: between pose x_0 and landmark ℓ_1
  - $y^{rb}_{1,1}$: between pose x_1 and landmark ℓ_1
  - $y^{rb}_{2,1}$: between pose x_2 and landmark ℓ_1
- Odometry measurements $y^{o}_{i,i+1}$ between successive poses.
- Loop closure constraint as odometry-like measurement $y^{lc}_{0,2}$ between pose states x_0 and x_2.



We now consider a more complex scenario. We will use the robot trajectory and landmarks in Figure 1. You will see three data files on Canvas. In the file `robot_history_5.csv`, we provide the ground truth history of the robot pose and measurements for a maximum sensor range of 5 meters. We will add noise and reconstruct the robot pose and landmark locations. The following steps will guide you in constructing the factor graph in GTSAM.

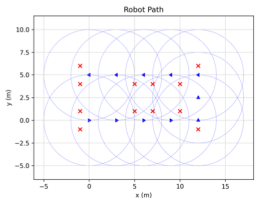


Figure 1: Robot Ground Truth Path. The landmarks are indicated as a red '×', the robot is a blue triangle oriented in the direction of travel, and the dotted blue circles indicate the maximum sensor range.

Please refer to the provided Google Colab at the link below (using your Stanford login), though you do not need to use Google Colab if you prefer to install GTSAM locally.

https://colab.research.google.com/drive/1tJqy1jNUeGC2V235yTdWqyOElQdIclMp?usp=sharing

(b) Verify that you can pip install GTSAM. On Google Colab, you can run `!pip install gtsam` at the top. Please read through the provided code to load and process the files. Instantiate the factor graph with:

```
graph = gtsam.NonlinearFactorGraph()
```

This is an empty factor graph. To *add* a factor to the factor graph, we will call `graph.add(...)`. To simplify the notation, GTSAM offers shorthand variables `X` and `L` for state and landmark, respectively, to organize the SLAM factors. In the provided Google Colab, we have initialized lists for `X` and `L`. The first factor we want to add is the prior factor to initialize the map at the origin. Fill in the required code based on the factor description below.

```
gtsam.PriorFactorPose2(X_start, pose, noise)
```

For `X_start`, use the zero-th element of the `X` list. For `pose`, use the `gtsam.Pose2(x, y, theta)` function specifying the origin and zero angle (i.e., aligned with the x-axis).

(c) Iterate through the provided data (`Pandas` is recommended). Use the factor type below to add the relative motion factors (i.e., from the odometry or relative state between successive states).

```
gtsam.BetweenFactorPose2(X_prev, X_curr, rel_pose, motion_noise)
```

Use the factor type below to add the bearing and range factors (i.e., from the sensor model) per each landmark in view at the given time step.

```
gtsam.BearingRangeFactor2D(X_curr, L_curr, bearing, range, noise
```

For the bearing measurement, use `gtsam.Rot2.fromDegrees(bearing.angle)`. Lastly, we will use the `BetweenFactorPose2` method to add a loop closure constraint between the second and eighth elements (zero-indexed) of the `X` list with position offset of 0 horizontally, 5 vertically, and 180 degrees in rotation. How many factors are in your factor graph (print `graph` to verify)?

(d) We will initialize the values from the motion alone. Instantiate the initial estimate with

```
initial_estimate = gtsam.Values()
```

You can input an initial estimate with `initial_estimate.insert(X_or_L_var, pose_or_point)`. Be sure to use `gtsam.Pose2(x, y, theta)` for the pose and `gtsam.Point2(x, y)` for the landmark. Plot the initial estimates for `X` and `L` using the functions below:

```
gtsam_plot.plot_pose2(...) and gtsam_plot.plot_point2(...).
```

Include the plot in your write-up.

(e) Run the Levenberg Marquardt in GTSAM as listed in the Google Colab. GTSAM will provide estimates of `X` and `L` through the optimizer and will provide the marginal Gaussian distributions in `gtsam.Marginals`. Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma). Include the plot in your write-up.

(f) What do you notice about the orientation of the solution and the shape and orientation of the confidence ellipses? Why does this occur?

(g) Switch out `robot_history_5.csv` for `robot_history_3.csv` (maximum sensor range of 3 meters) and `robot_history_10.csv` (maximum sensor range of 10 meters). Using the same initialization for `X` and `L`, reconstruct the factor graphs. What are the factor graph sizes in each case?
Hint: You should only have to change the file input. All the other functions you wrote should still be applicable!

(h) Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma) in each case. What do you notice about the final results at different sensor ranges? Explain your result with the connectivity of the resulting factor graphs.

c) 57 factors

```
NonlinearFactorGraph: size: 57

Factor 0: PriorFactor on x0
  prior mean:  (0, 0, 0)
  noise model: diagonal sigmas [0.1; 0.1; 0.0174532925];

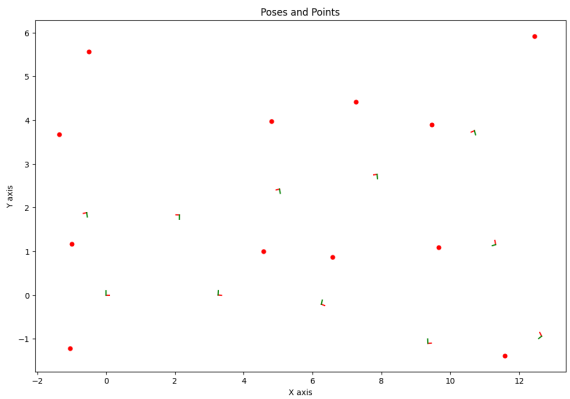
Factor 1: BetweenFactor(x0,x1)
  measured:  (3.25492435, 0, -0.0706048005)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 2: BearingRangeFactor
Factor 2:   keys = { x1 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.89661399
range  4.12310563

Factor 3: BearingRangeFactor
Factor 3:   keys = { x1 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

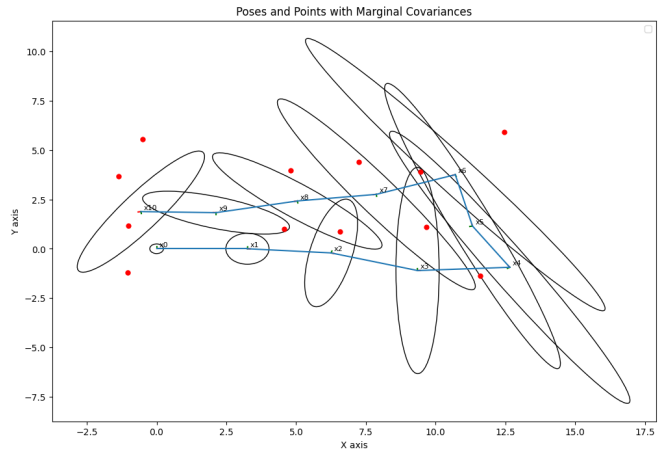
Factor 4: BearingRangeFactor
Factor 4:   keys = { x1 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
...
  measured:  (-0.233879407, 1.95151853, 2.97021382)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];
```

d)

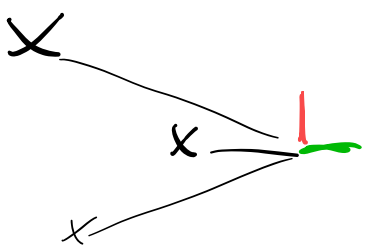


• Landmarks

e)



f) The covariance is lower when there are landmarks that are all in a line, which can provide more distance information in that direction.



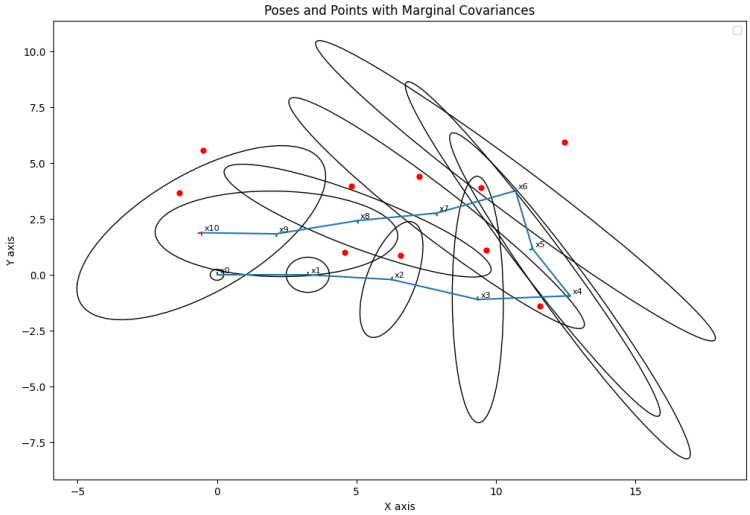
More information in given direction to decrease uncertainty

g)

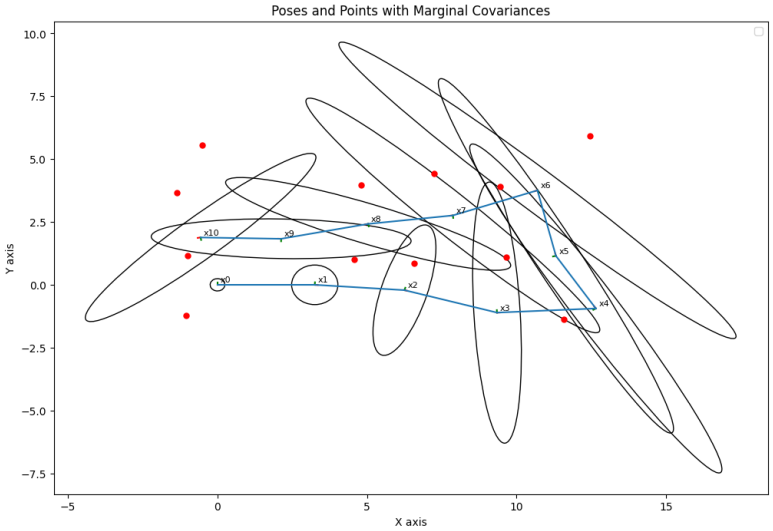
Factor graph sizes

3m → 30 nodes
5m → 57 nodes
10m → 106 nodes

3



10



h) For poses that are far from landmarks, like with 9 and 10, increasing from 3m to 10m included several additional sensors particularly in the direction along the "short" direction of the ellipses. More measurements/ factors on the factor graph allow optimizer to have more constraints and have less uncertainty